



Lecture 11 – Reinforcement Learning (RL): Finite MDP

Advanced Analytics and Applications [AAA]

Advanced Analytics and Applications

Teaching Strategy

	Advanced ML	Deep Learning	Simulation	Reinforcement Learning
What	Introduction + Recap Machine Learning Kernel Machines (Classification) - Support Vector Machines - Kernels Soft Clustering - Hard Clustering - Soft Clustering (GMM) Spatial Analytics - Reference Systems (WGS84) and Geometry Objects - Spatial Data Science	Introduction to Neuronal Networks (MD) Deep Forward Networks (Stochastic) Gradient Descent - Backpropagation Regularization and Optimization for NN - Dropout - Penalties - Early Stopping etc. Special Architecture NNs An outlook into what is used in practice, recent developments	Simulation Discrete Event Simulation, ABM	Introduction to Reinforcement Learning - Motivation + History - Multi-Armed Bandits Tabular Methods - Finite Markov Decision Processes - Dynamic Programming - RL Function Approximation and Q-Learning Policy Gradient Methods and Applications - Q & A Session / Mock Exam
When	15.04., 22.04., 29.04., 06.05.	13.05., 20.05., 20.05., 03.06.	10.06.	17.06., 01.07., 08.07., 15.07., 22.07.

I Finite Markov Decision Process

II Planning by dynamic programming (model-based)

III Learning by RL (model-free)

Sequential Decision Making



- How can I reach the center with minimum moves?
- Does my current decision affect the next decision?
- How to solve such problems?
 - Today, we talk about it.

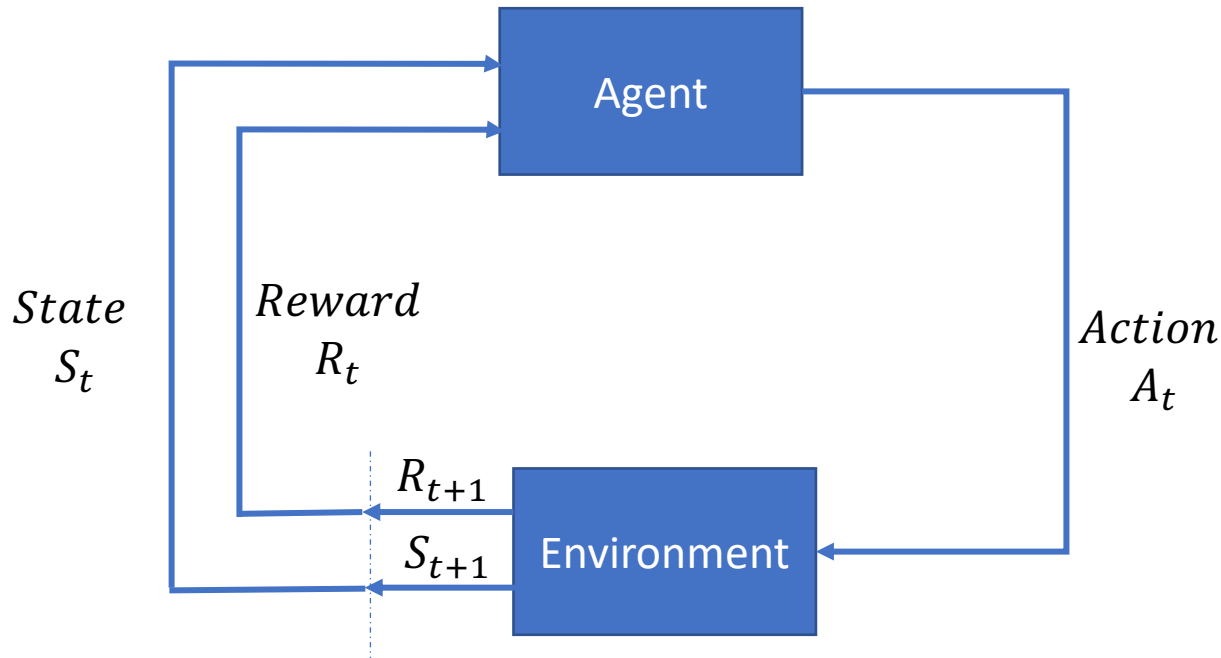
Introduction on MDPs

- To solve dynamic and sequential problems, we need the Markov Decision Process (MDP).
- What is Markov Decision Process (MDP)?
- MDPs are mathematical formulation of **sequential decision-making** problems,
 - where actions influence immediate rewards, subsequent states, and future rewards
- The role of MDPs in RL
 - Markov decision processes formally describe an environment for reinforcement learning,
- Almost all RL problems can be formalised as MDPs, e.g.
 - Optimal control primarily deals with continuous state space
 - MDPs Partially observable problems can be converted into MDPs
 - Bandits are MDPs with one state

Markov process

- A Markov process is a **memoryless** random process, i.e., a sequence of random states $S_1, S_2, \dots$ with the Markov property.
 - **Markov property**: Current state completely characterises the state of the world
- A Markov Process (or Markov Chain) is a tuple $\langle S, A, P, R \rangle$
 - S is a set of **states**
 - A is a set of **actions**
 - P is a state **transition probability** function, $(P_{ss'}^a = P[S_t = s' | S_{t-1} = s, A_{t-1} = a])$
 - R is a **reward** function of states

Agent and environment in MDPs



- MDP is a straightforward framing of the problem of learning from **interaction** to achieve a **goal**.
- Learner and decision maker is the **agent**.
- Anything that cannot be changed arbitrarily by the agent is outside of it and thus part of its **environment**
- Interacting at each of a sequence of discrete time steps
- At each time step, $S_t \in \mathcal{S}$, $A_t \in \mathcal{A}(s)$, $R_t \in \mathcal{R}$
- In finite MDP, sets of actions, states, and rewards have a finite number of elements

State transition probabilities

- Discrete probability distribution of S_t and R_t :

$$p(s', r|s, a) = \Pr\{S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a\}$$

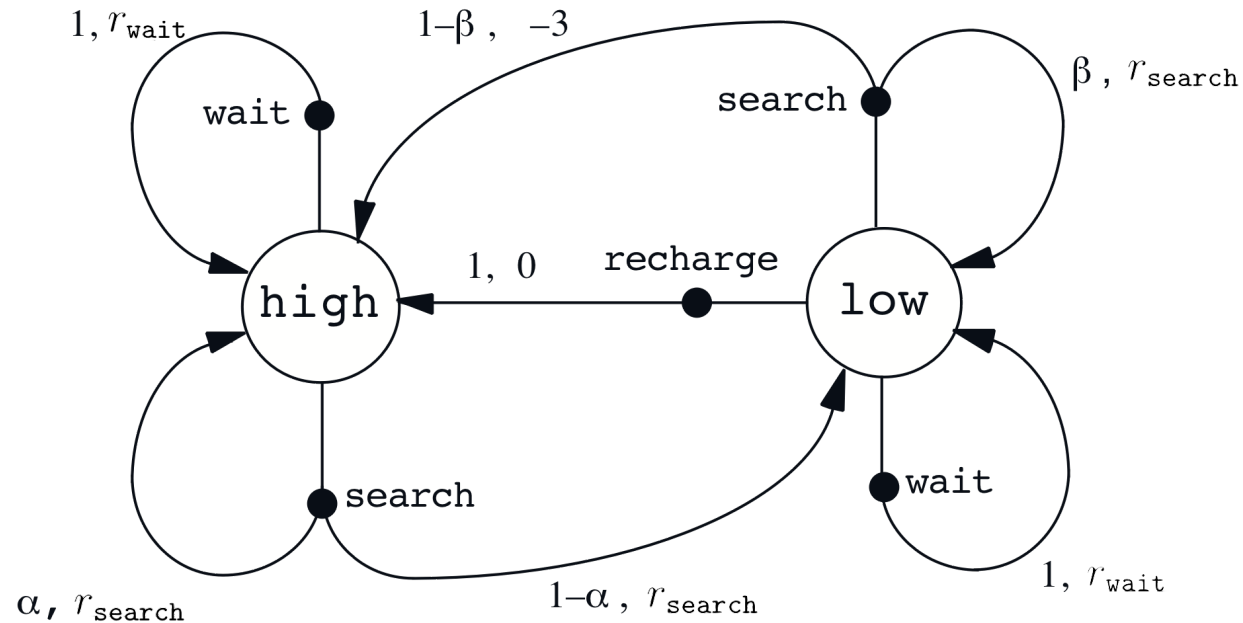
- This is an ordinary deterministic function describing the environment.
- State transition probabilities:

$$p(s'|s, a) = \Pr\{S_t = s' | S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r|s, a)$$

- Expected reward for state-action pairs:

$$r(s, a) = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a)$$

An example (A Chargeable Recycling Robot)



s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	r_{wait}
low	wait	high	0	r_{wait}
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	0

- $\mathcal{S} = \{high, low\}$
- $\mathcal{A}(high) = \{search, wait\}$
- $\mathcal{A}(low) = \{search, wait, recharge\}$

Goals and Rewards

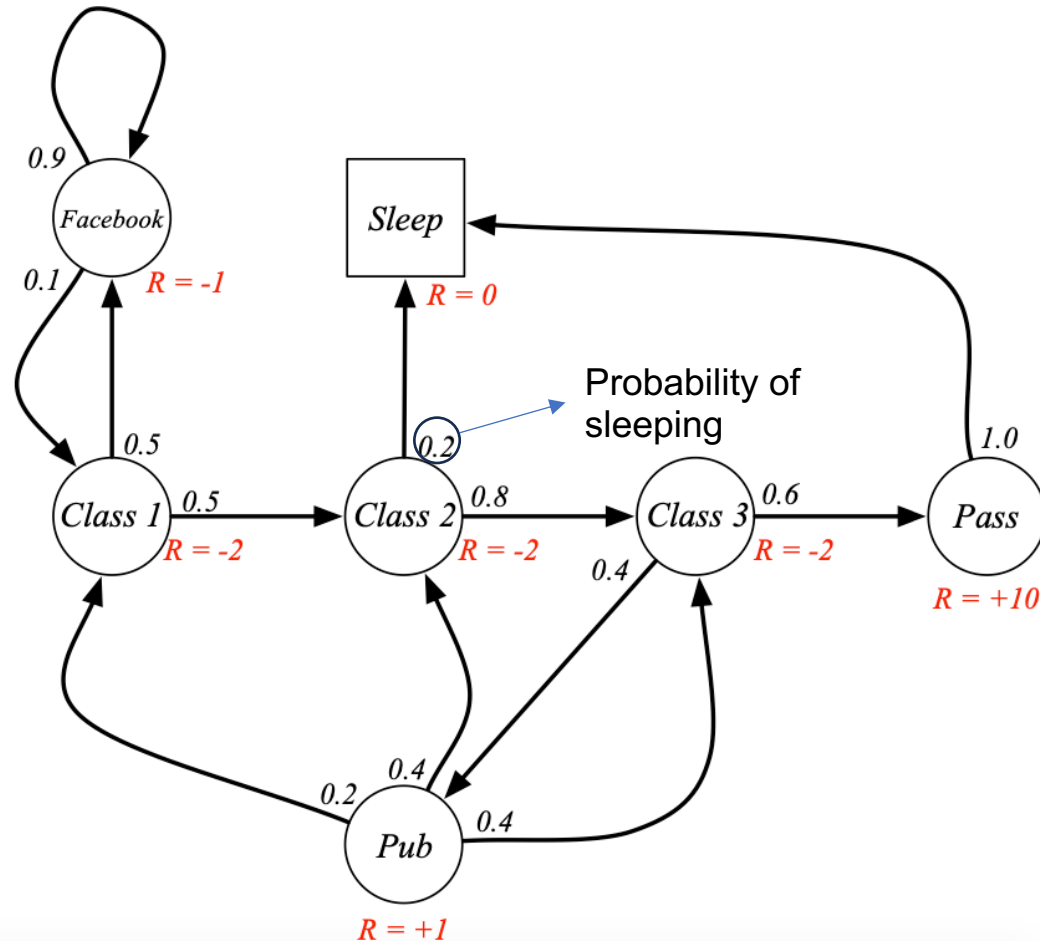
- Designing effective rewards is a critical part of RL problems.
- **Reward (R_t)**: Special signal (simple number) passing from environment to the agent
- **Goal**: Maximizing not immediate reward, but cumulative reward (expected **return**) in the long run

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

This formulation is for episodic tasks

- **Episode**: Subsequences of agent-environment interaction
 - Each episode ends in a special state, called the terminal state
 - The next episode begins independently of the previous one

Example of episodic Markov chain



- Starts from Class 1
- Ends to Sleep (terminal state)

Sample episodes

- C1 C2 C3 Pass Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep

Return in Continuing tasks

- The interaction cannot break into identifiable episodes (no terminal states), so the proposed return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- γ is discount rate, determining the present value of future rewards ($0 \leq \gamma \leq 1$)
 - If $\gamma < 1$, G_t has a finite value.
 - If $\gamma = 0$, the agent is myopic
 - As γ approaches 1, the agent become more farsighted

Why is Markov process discounted?

- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards



Suppose $\gamma = 0.5$ and the following sequence of rewards is received $R_1 = -1, R_2 = 2, R_3 = 6, R_4 = 3$ and $R_5 = 2$, with $T = 5$. What are $G_0, G_1, \dots, G_5$? (Hint: Work backwards)

- a) 2.5, 3.5, 5.5, 4, 2, 1
- b) 5.5, 7.5, 8.5, 4, 2, 0
- c) 4.5, 6.5, 6.5, 2.5, 0, 0

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$



Suppose $\gamma = 0.5$ and the following sequence of rewards is received $R_1 = -1, R_2 = 2, R_3 = 6, R_4 = 3$ and $R_5 = 2$, with $T=5$. What are $G_0, G_1, \dots, G_5$?

- a) 2.5, 3.5, 5.5, 4, 2, 1
- b) 5.5, 7.5, 8.5, 4, 2, 0
- c) 4.5, 6.5, 6.5, 2.5, 0, 0

$$G_5 = 0$$

$$G_4 = 2$$

$$G_3 = 0.5G_4 + 3 = 4$$

$$G_2 = 0.25G_4 + 0.5G_3 + 6 = 8.5$$

$$G_1 = 0.125G_4 + 0.25G_3 + 0.5G_2 + 2 = 7.5$$

$$G_0 = 0.0625G_4 + 0.125G_3 + 0.25G_2 + 0.5G_1 - 1 = 5.5$$

Policies in MDPs

- Almost all RL problems involve estimate value functions (state and/or state-action pairs)
- The value function $v(s)$ gives the long-term value of state s with respect of **policies**
- A policy π is a distribution over actions given states,

$$\pi(a|s) = \mathbb{P}[A_t = a | S_t = s]$$

- It defines the agent's behavior
 - MDP policies depend on the current state (not the history)
- Reinforcement learning methods specify how the agent's policy changes as a result of its experience.

Value functions

The **state value function** is the expected return starting from state s , and then following policy π

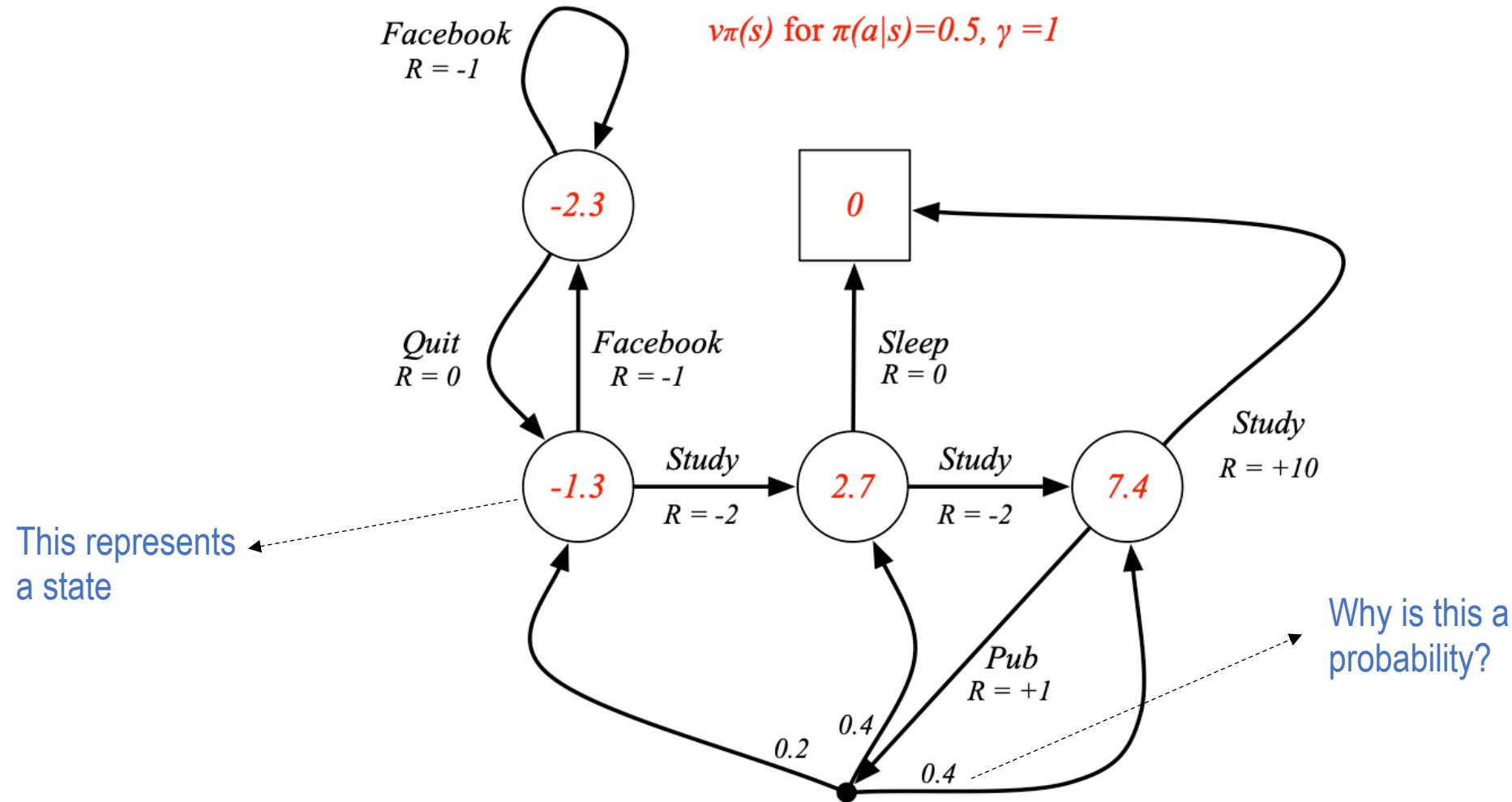
$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right]$$

The **action-value function** is the expected return starting from state s , taking action a , and then following policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a\right]$$

- The value functions v_{π} and q_{π} can be estimated from experience.

Example of state value functions



Bellman expectation equation

- The state-value function can be decomposed into immediate reward plus discounted value of successor state,

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) [r(s, a) + \gamma v_{\pi}(s')]$$

- It expresses a relationship between the value of a state and the values of its successor states.
 - For each a, r, s' , the probability is $\pi(a|s) = p(s', r|s, a)$
 - The probability multiplies to its return
 - Then sum over all possibilities
- The action-value function can similarly be decomposed,

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$



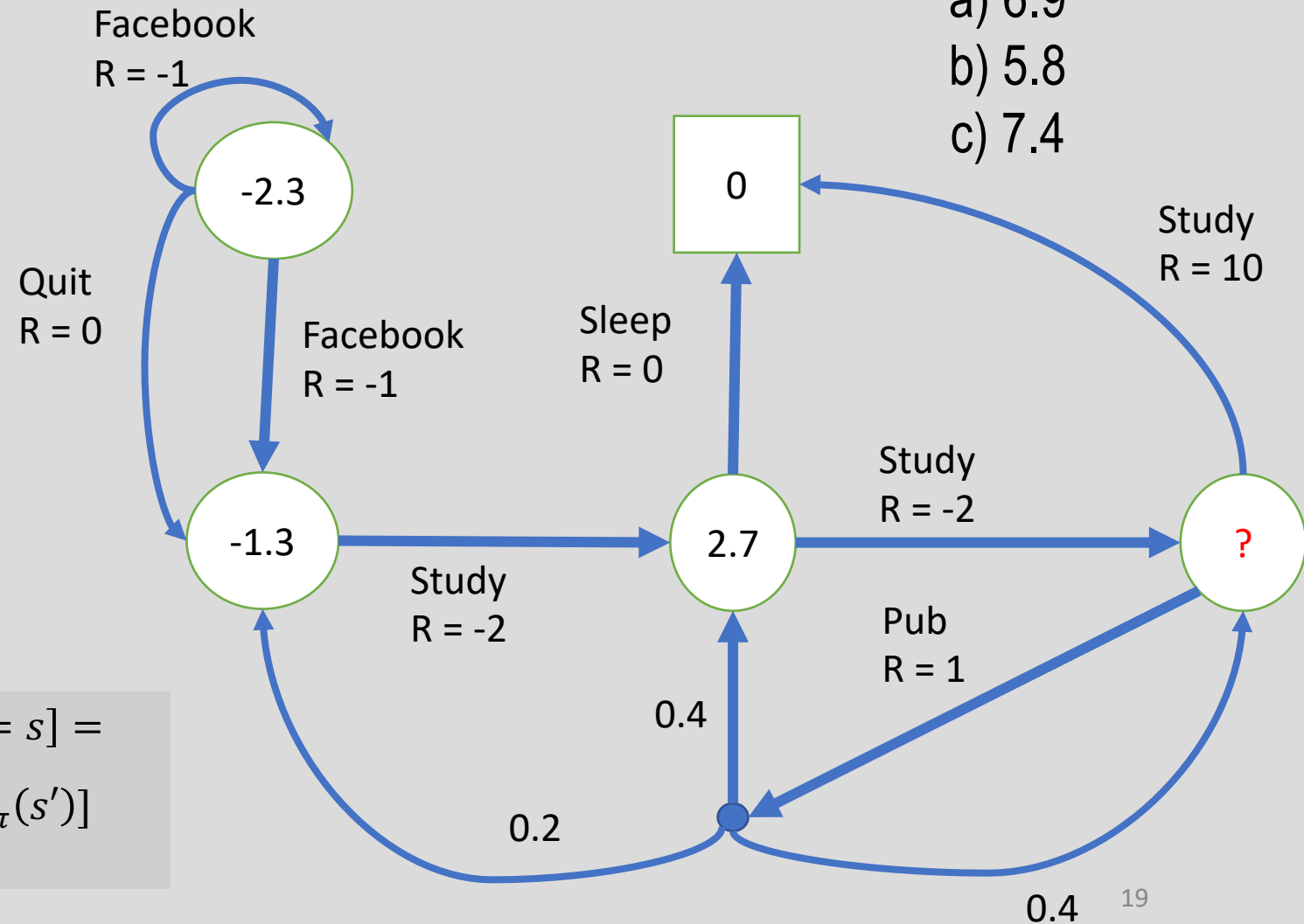
What is the asked (?) state value function with :
 $\pi(a|s) = 0.5$ for all states and actions, $\gamma = 1$

Suppose random initialization of (?) to be 5.0

a) 6.9

b) 5.8

c) 7.4



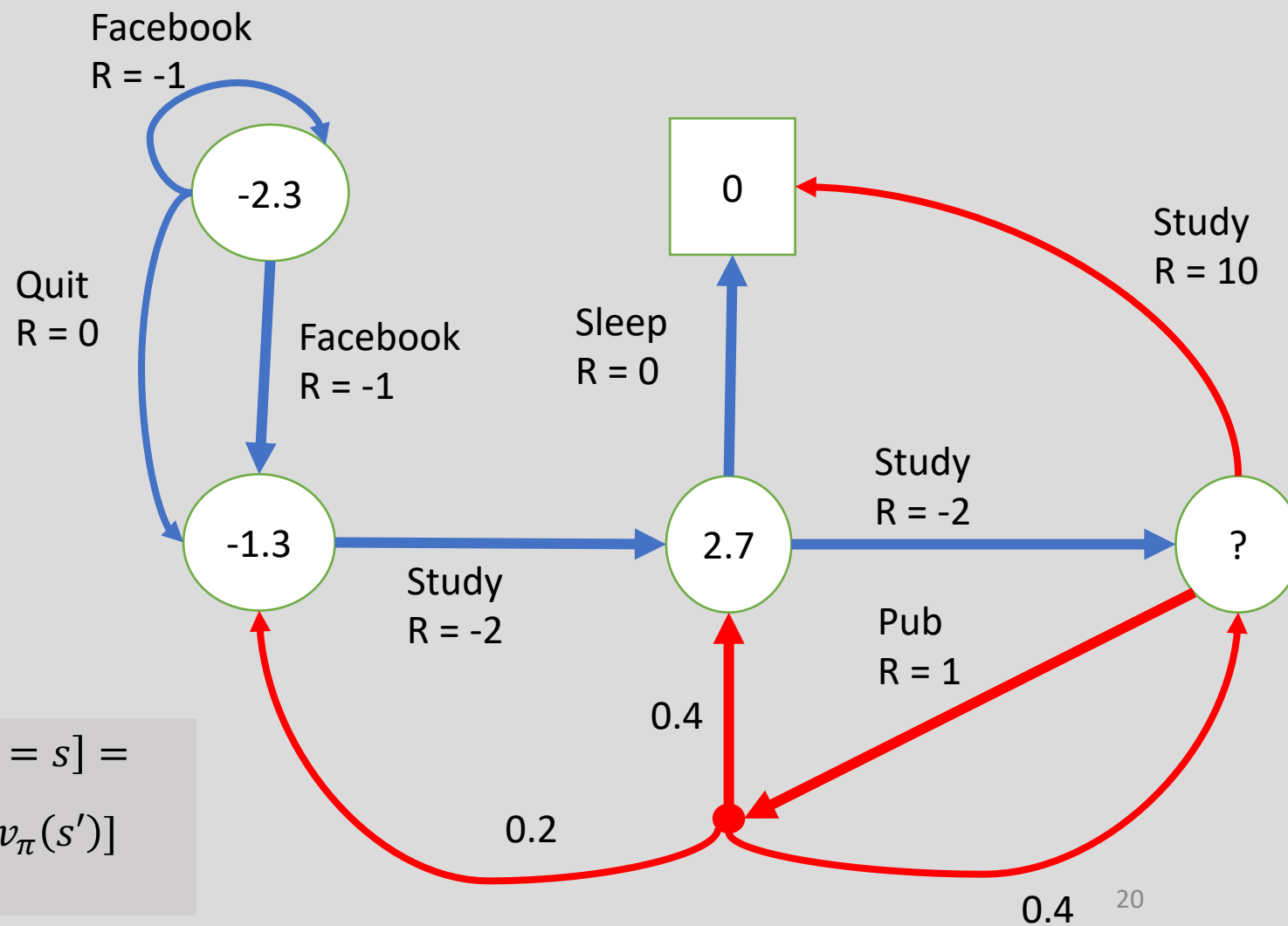
$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$



$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

$$V_{\pi} = 0.5 * (1 + 0.2 * -1.3 + 0.4 * 2.7 + 0.4 * 5.0) + 0.5 * 10$$

$$V_{\pi} = 6.91$$



Solving an MDP: optimal policy

- Solving a MDP means, roughly, finding a policy that achieves a lot of reward over the long run.

Defines a partial ordering over policies

$$\pi \geq \pi' \text{ if } v_\pi \geq v_{\pi'} \quad \forall s$$

For any Markov Decision Process:

- There is at least an optimal policy π_* that is better than or equal to all other policies,

$$\pi_* \geq \pi, \quad \forall \pi$$

- All optimal policies achieve the optimal value function,

$$v_{\pi_*}(s) = v_*(s)$$

- All optimal policies achieve the optimal action-value function,

$$q_{\pi_*}(s, a) = q_*(s, a)$$

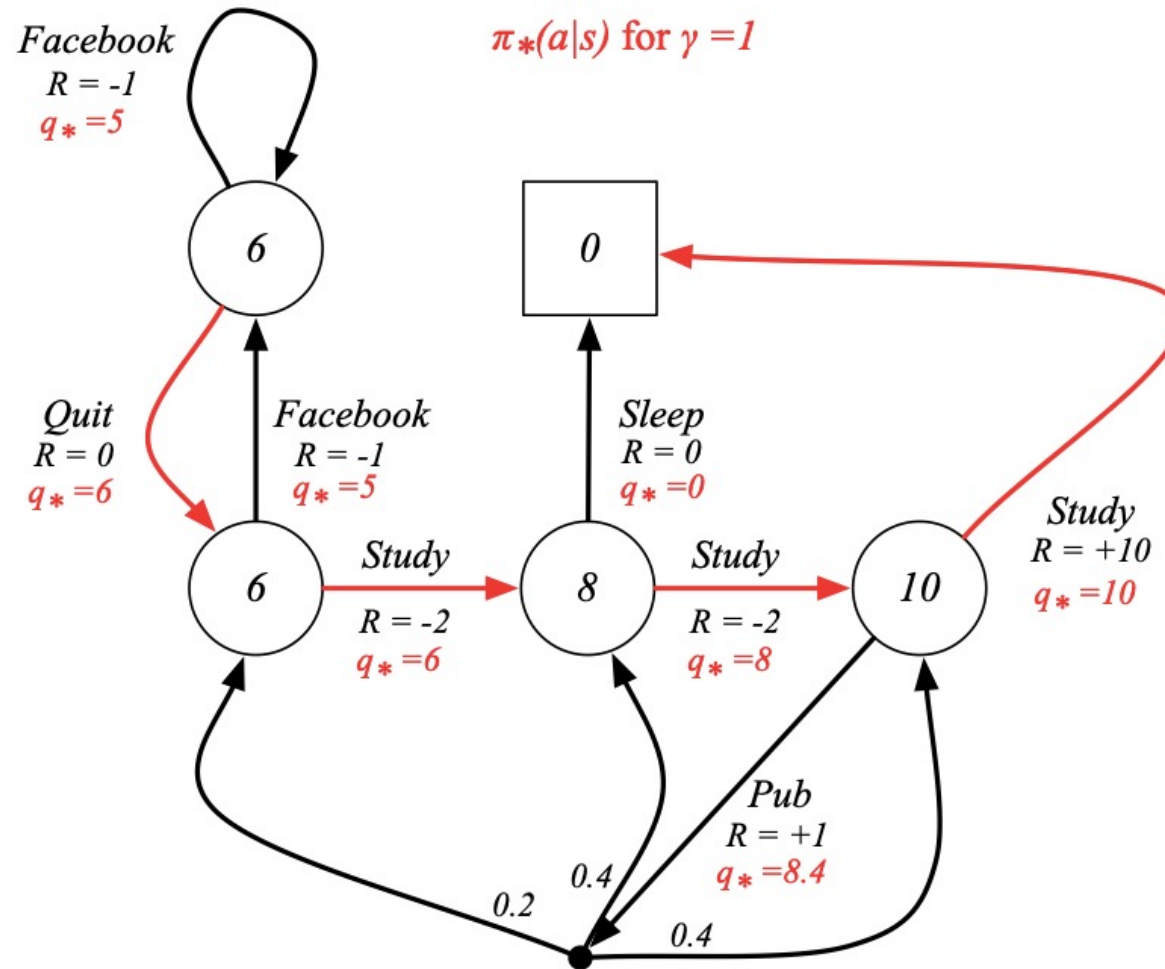
Finding an optimal policy

- An optimal policy can be found by maximising over $q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in A}{\operatorname{argmax}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, we immediately have the optimal policy

Example: Optimal policy for Student MDP



Optimal value function

- The optimal state-value function $v_*(s)$ is the maximum value function over all policies

$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

- The optimal action-value function $q_*(s, a)$ is the maximum action-value function over all policies

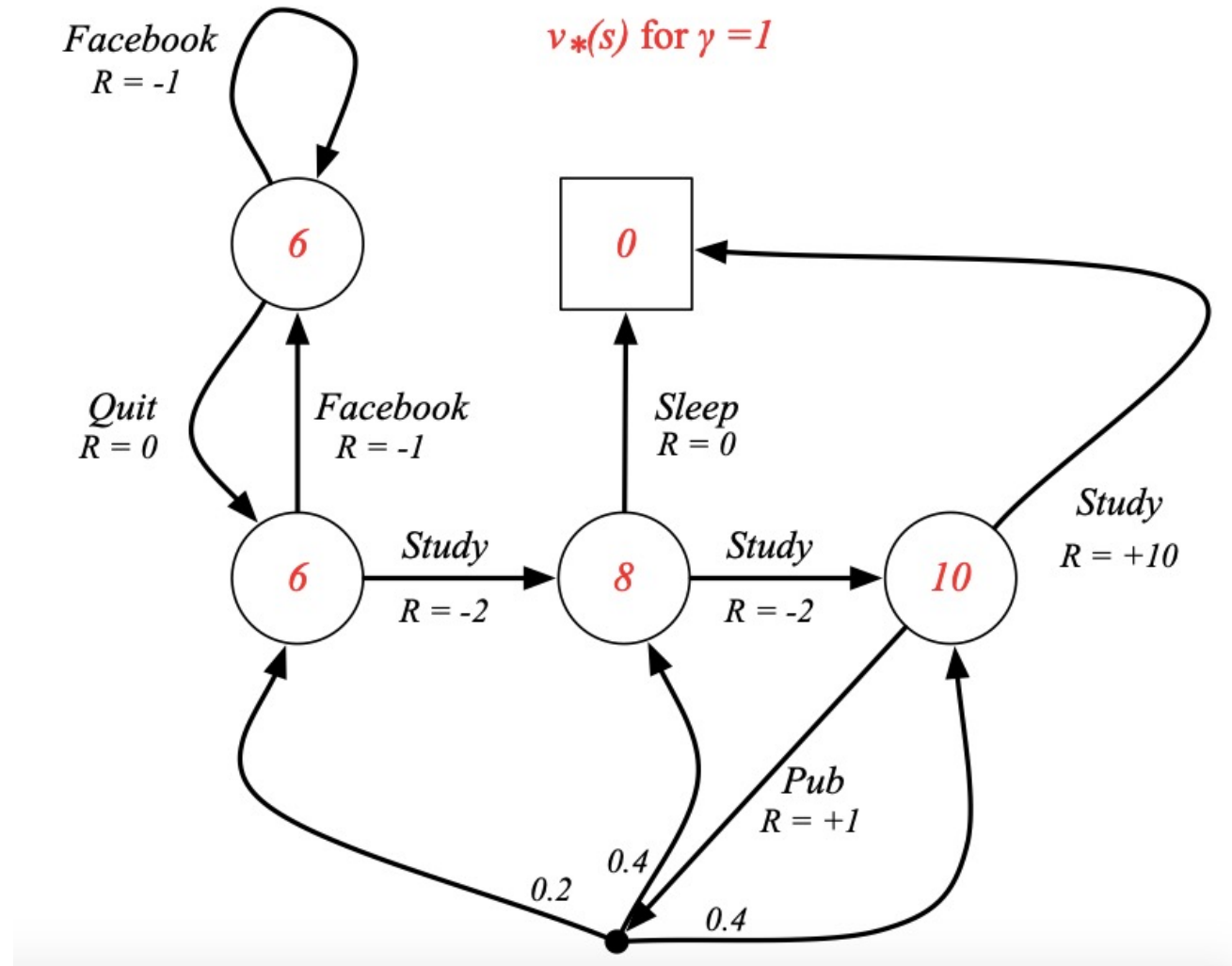
$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

- For the state-action pair (s, a) , this function gives the expected return for taking action a in state s and thereafter following an optimal policy.

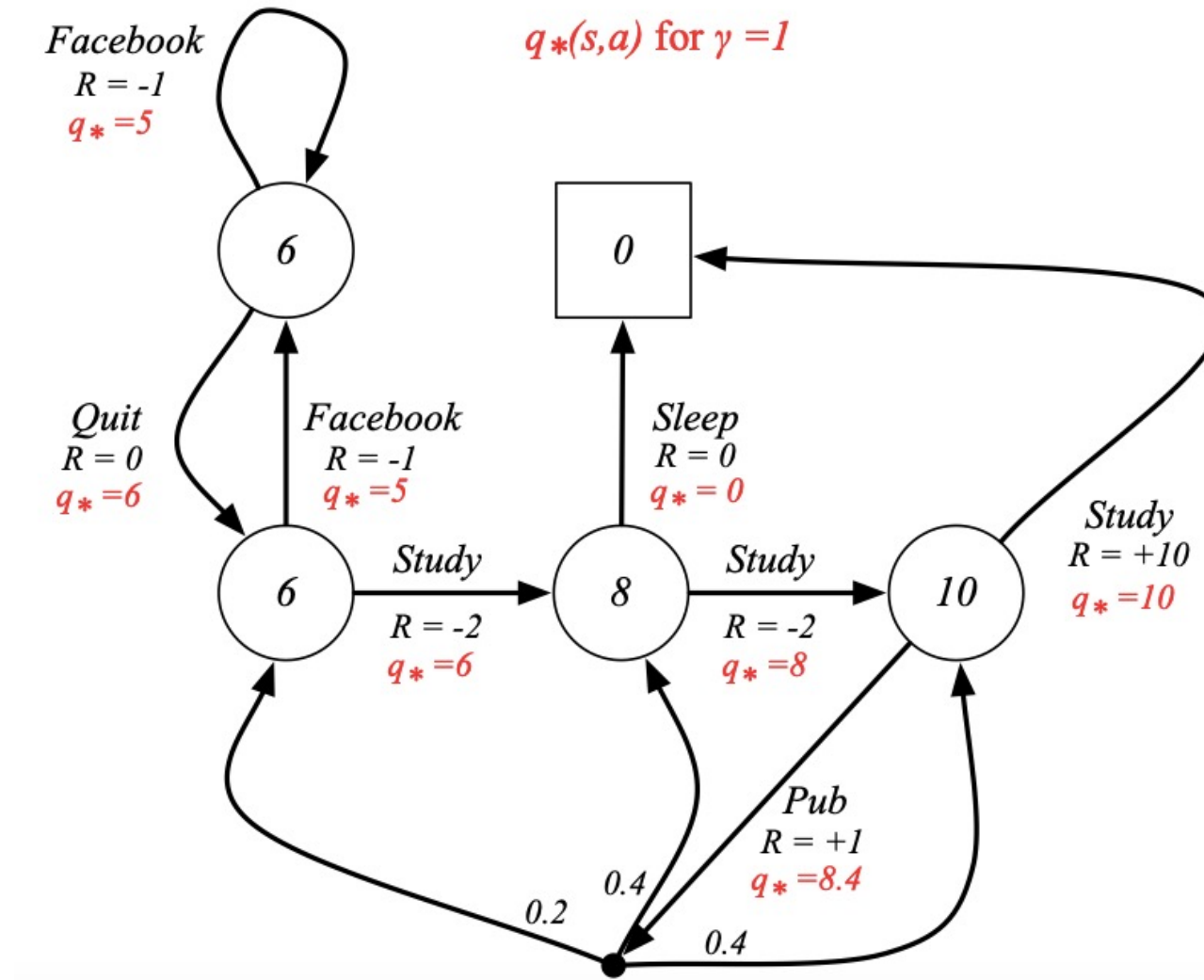
$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

- The optimal value function specifies the best possible performance in the MDP.
- An MDP is “solved” when we know the optimal value function.

Example: Optimal State Value Function for Student MDP



Example: Optimal Action-Value Function for Student MDP



Bellman optimality state equation

- v_* and q_* must satisfy Bellman equation function and can be written without reference to any specific policy.
- The value of a state under an optimal policy must equal the expected return for the best action from that state:

$$\begin{aligned} v_*(s) &= \max_{a \in A(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}_{\pi_*}[G_t | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \end{aligned}$$

Bellman optimality state-action equation

- Bellman optimality action equation

$$\begin{aligned} q_*(s, a) &= \mathbb{E}[R_{t+1} + \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')] \end{aligned}$$

Unique solution

- Bellman optimality equation has a **unique** solution **independent** of the policy

$$v_*(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')]$$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a) [r + \gamma \max_{a'} q_*(s',a')]$$

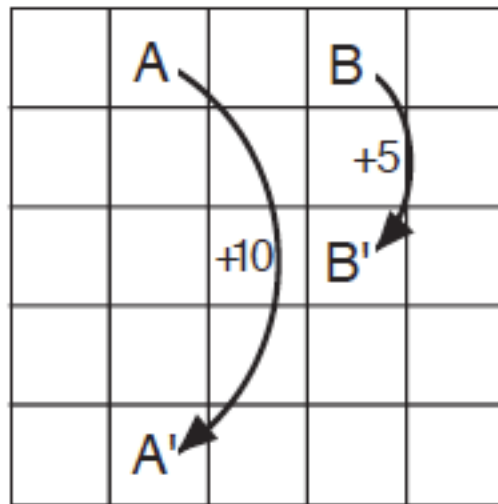
- This is a **system** of equations; if there are **n states**, then there are **n equations** in **n unknowns**
- If the dynamics p of the environment are known, this system can be solved using variety of methods

Determine optimal policy using v_* and q_*

- Once one has v_* ,
 - For each state, there will be **one or more actions** at which maximum is obtained
 - Any policy that assigns nonzero probability only to these actions is an optimal policy (**one-step search**)
 - With v_* , actions that appear best after a one-step search will be optimal actions
 - **Greedy** policies with respect to v_* are optimal policies (**the beauty of v_***)
- Once one has q_* ,
 - The agent does not even have to do a one-step-ahead search
 - For each state, the optimal action is the one that maximizes $q_*(s, a)$
 - If the agent does not know the environment's dynamics, it can find optimal actions

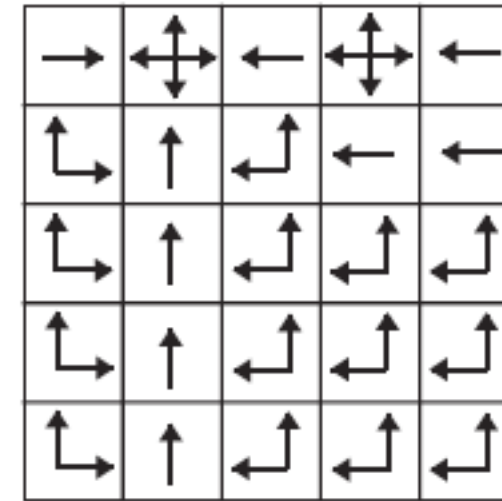
Solving a GridWorld: an example

- Each movement has a negative reward (-1), continues, $\gamma = 0.9$



Gridworld

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

 v_*  π_*

Is it possible to solve MDPs optimally?

- This optimal solution relies on at least three assumptions which rarely happens:
 - The environment's dynamics are accurately known
 - Enough computational resources
 - Markov property
- For example, the game of backgammon has about 10^{20} states and takes thousand of years to solve Bellman equation for v_*
- Many different methods can approximately solve the Bellman optimality equation
 - Heuristic search methods
 - Approximate dynamic programming
 - Reinforcement learning methods

Optimality and Approximation

- An agent that learns an optimal policy has done very well, but in practice this rarely happens.
- Mostly, limited computational power leads to using approximation
- Available memory is a critical constraint:
 - For small and finite state sets, we can form these approximations using **arrays or tables** (tabular cases)
 - In many cases, **functions** must be approximated
- Key property that distinguish RL from other approaches to approximately solving MDPs:
 - Online nature of RL makes it possible to put more effort into learning to make good decisions for frequently encountered states, at the expense of less effort for infrequently encountered states.
- Many iterative solution methods
 - Value Iteration
 - Policy Iteration
 - Q-learning
 - SARSA

Solving MDPs

- **Model-based** methods (e.g., Dynamic Programming)
 - Solve a fully known MDP
 - The mathematical model of the environment is available
 - This is a **planning** problem
- **Model-free** Reinforcement learning (e.g., Monte Carlo, Temporal Difference)
 - The mathematical model of the environment is **NOT** available
 - The agent interact with the environment to find the optimal policy
 - This is a **learning** problem

Methods

Solution for Finite Markov Decision Problems (MDP)

Use iterative methods in order to compute optimal policy that maximize the state value function

1 Dynamic Programming (DP)

- Require a complete model of the environment
- Well-developed mathematically
- Guaranteed optimal policy
- Not suitable for large problems
- No sampling, no bootstrapping

$$V(S_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma V(S_{t+1})]$$

2 Monte Carlo (MC)

- Learn directly from complete episodes of experience
- Don't require a model
- Conceptually simple
- Only suitable for episodic MDPs
- Sampling, no bootstrapping

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

3 Temporal-difference (TD)

- Require no model
- Learn from incomplete episodes
- Fully incremental
- More complex to analyse
- Sampling and bootstrapping

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

Finite Markov Decision Process

Planning by dynamic programming (model-based)

Learning by RL (model-free)

Introduction on Dynamic Programming

Dynamic:

- sequential or temporal component to the problem

Programming:

- optimising a “program”, i.e. a policy
- A method for solving complex problems
- By breaking them down into subproblems
 - Solve the subproblems
 - Combine solutions to subproblems
- DP assumes full knowledge of the MDP and is used for **planning** in an MDP

We have two different problems to solve: Prediction and Control

- **Prediction** problem:

- **Estimating** the value functions for a given policy
- Input: A MDP and policy π
- Output: value functions v_π and/or q_π

- **Control** problem:

- Finding **optimal** policy/policies for MDP problems
- Input: MDP environment
- Output: Optimal policy π_* and value functions v_* , q_*

Policy evaluation: prediction

- Problem: evaluate a given policy
- Solution: Iterative computation of the value functions for a given policy ($V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_\pi$)

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

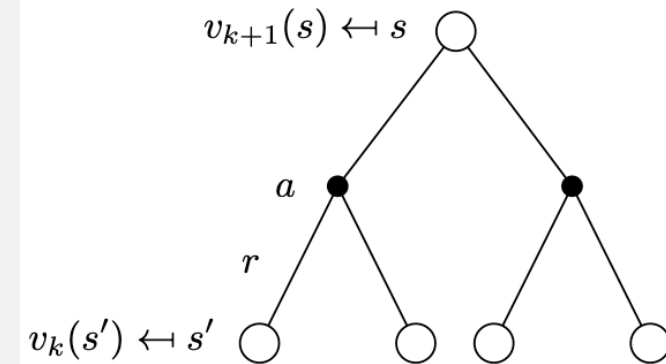
Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

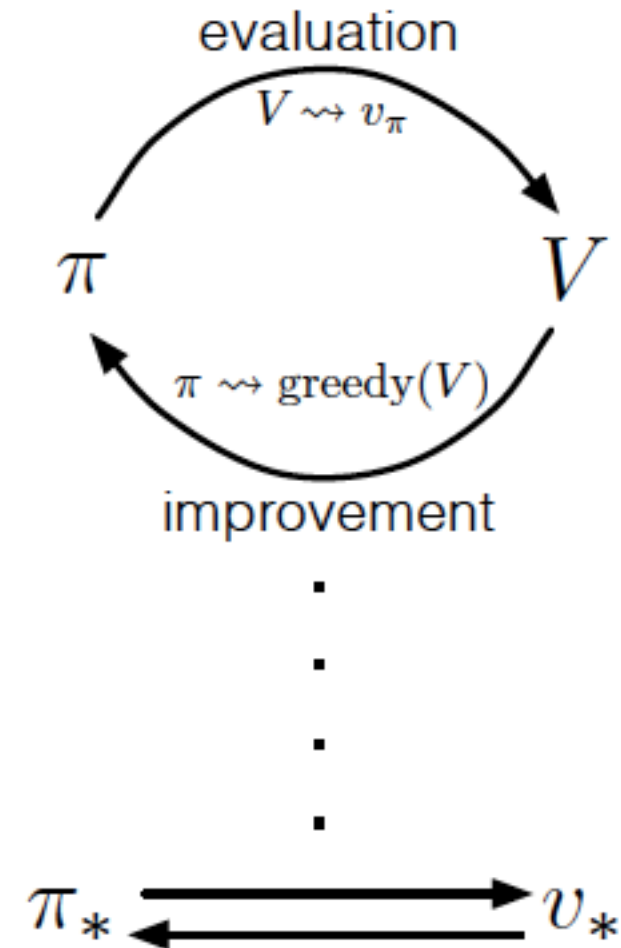
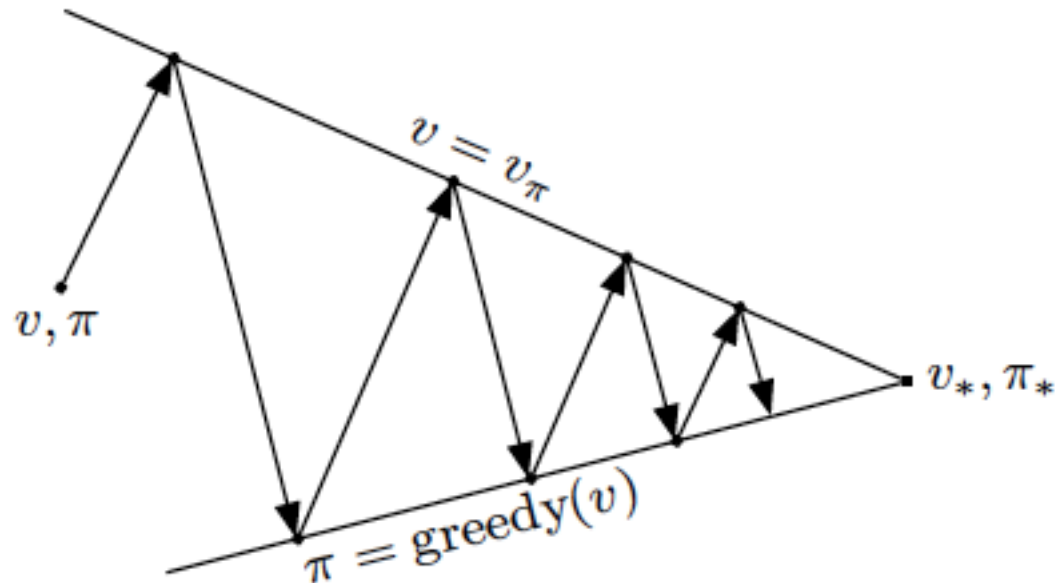


Policy improvement: control

- Next step: Improve our policy
- Problem: Computation of an improved policy given the value function for that policy
- Solutions: Given a policy π ,
 - Evaluate the policy π
 - $V_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$
 - Improve the policy by acting greedily with respect to V_{π}
 - $\pi' = greedy(V_{\pi})$
- Putting these together, we have policy iteration and value iteration

Policy iteration

- Once π' is obtained, we can compute $v_{\pi'}$ and then π'' .
- $\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_{\pi_*}$,
- This way of finding an optimal policy is called **policy iteration**.



Policy iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ *true*

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Example of Policy Iteration

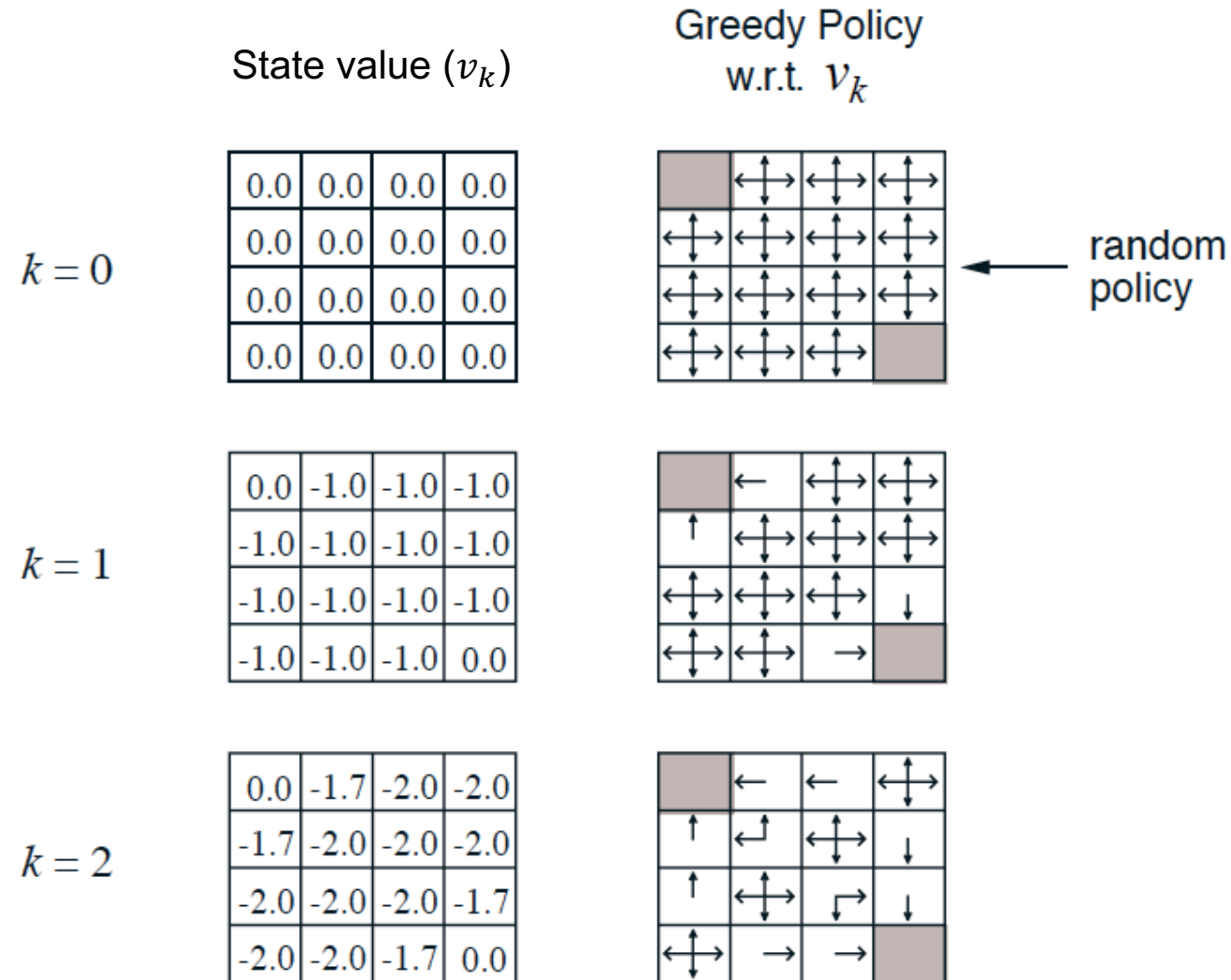
	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

- Undiscounted episodic MDP ($\gamma = 1$)
- Terminal states (shown as shaded squares)
- Actions: north, south, east, west
- Actions leading out of the grid leave state unchanged
- Reward is -1 until the terminal state is reached
- Agent follows uniform random policy

$$\pi(n|.) = \pi(s|.) = \pi(e|.) = \pi(w|.) = 0.25$$

We show the process in the following slide.

Example of Policy Iteration



Example of Policy Iteration (cont..)

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

	←	←	↙
↑	↖	↙	↓
↑	↗	↘	↓
↘	→	→	

	←	←	↙
↑	↖	↙	↓
↑	↗	↘	↓
↘	→	→	

	←	←	↙
↑	↖	↙	↓
↑	↗	↘	↓
↘	→	→	

optimal policy

Value iteration: another approach to solve MDPs

- Value iteration combines the policy improvement and policy evaluation steps:

$$\begin{aligned} v_{k+1}(s) &\doteq \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')], \end{aligned}$$

- Iterative application of Bellman optimality backup

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_*$$

- Using synchronous backups

- At each iteration $k + 1$
- For all states $s \in S$
- Update $v_{k+1}(s)$ from $v_k(s')$

- Unlike policy iteration, there is no explicit policy**

Value iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```

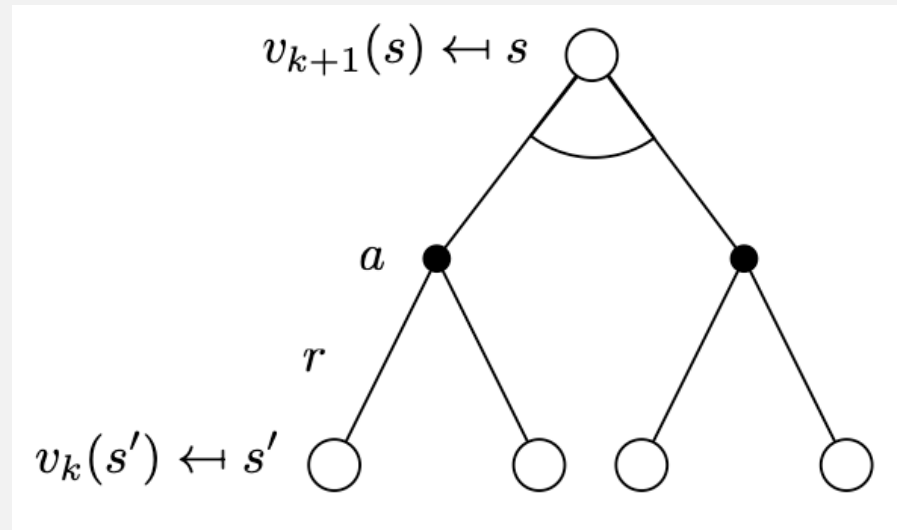
|  $\Delta \leftarrow 0$ 
| Loop for each  $s \in \mathcal{S}$ :
|    $v \leftarrow V(s)$ 
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 

```

**Main difference with
policy iteration**

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$



Policy evaluation is limited only to one iteration.

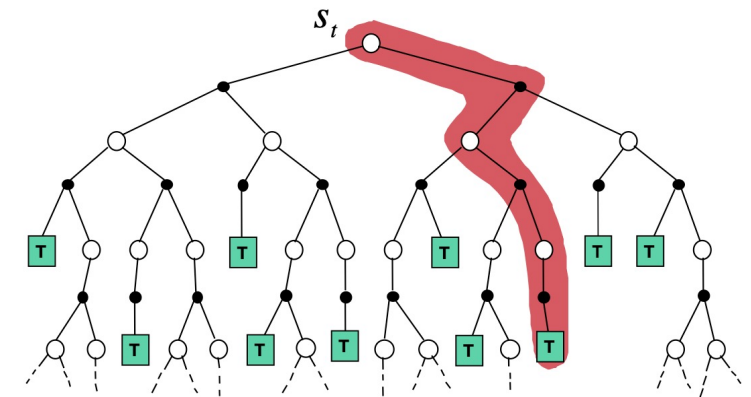
I Finite Markov Decision Process

II Planning by dynamic programming (model-based)

III Learning by RL (model-free)
Monte Carlo

Monte Carlo in RL

- Monte-Carlo (MC) is a **model-free** method
- Finding the value functions and optimal policies from **episodes of experience**
- It learns from **interaction** without any model of the environment
- MC can be used with **simulation**
- Learns from complete episodes: **no bootstrapping**
- Uses the simplest possible idea: **value = mean return**
 - $G_t = \sum_t \gamma^t r_t$ (until the end of the episode)
- Can only be applied to **episodic MDPs**
 - all episodes must terminate



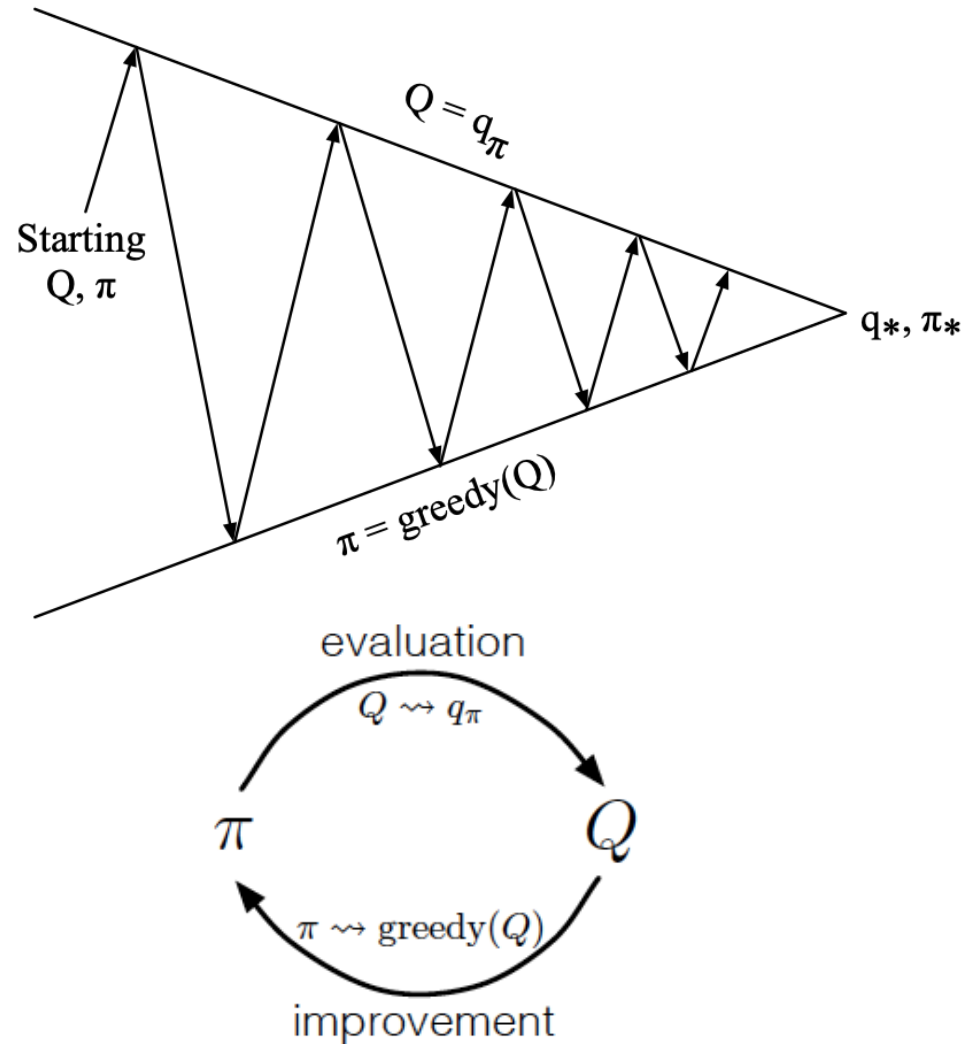
Monte Carlo Prediction

- Learning the state-value function for a given policy using MC
- Rather than using a model to compute the value of each state, it averages many returns that start in the state
- To evaluate state s :
 - *Initialize* $V(s)$ and $G(s) \quad \forall s \in \mathcal{S}$
 - *Loop forever (for each episode):*
 - *Generate an episode following* π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
 - *loop for each step of episode, $t = T - 1, T - 2, \dots, 0$:*
 - $G \leftarrow G + R_{t+1}$
 - Append G to the $\text{Returns}(S_t)$
 - $V(s) \leftarrow \text{average}(\text{Returns}(S_t)) : V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$
- Monte-Carlo policy evaluation uses **empirical mean return instead of expected return**

Policy Improvement: Estimating of action values

- For a control problem without a model:
 - We need to estimate action values (we don't know the dynamics of the environment)
 - State values alone are not sufficient to determine a policy
- Policy evaluation for action values
 - Estimate $q_{\pi}(s, a)$: Expected return starting in state s , taking action a and following policy π
- Challenge: many action-pairs may never be visited
 - Specifying that episodes starting in a state-action pair
 - Considering stochastic policies with nonzero probability of selecting all actions

Monte Carlo Control



- Overall idea is like DP generalized policy iteration
- We need **action-value (Q)** function
- Initialize $V(s)$ and $G(s)$, $\pi = e - \text{greedy}(Q) \forall s \in \mathcal{S}$
- Loop forever (for each episode k):
 - Generate an episode following π :
 - $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
 - loop for each step of episode, $t = T - 1, T - 2, \dots, 0$:
 - $G \leftarrow G + R_{t+1}$
 - Append G to the Returns(S_t)
 - $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[G_t - Q(s_t, a_t)]$
 - Update epsilon
 - Update policy: $\pi_k = e - \text{greedy}(Q)$

Summary and next steps

- This Lecture:
 - How to formulate a sequential decision-making problem using MDP
 - Transition probabilities
 - Policy and value functions
 - How to solve a MDPs using iterative methods
 - Planning by Dynamic programming
 - Policy and value iteration
 - Model-free prediction and control RL (Monte Carlo)
- Next lecture:
 - Model-free prediction and control RL (Temporal Difference (Q-learning))
 - Function Approximation
 - Applications

Contact



For general questions and enquiries on **research**, **teaching**, **job openings** and new **projects** refer to our website at www.is3.uni-koeln.de



For specific enquiries regarding this course contact us by sending an email to the **IS3 teaching** address at is3-teaching@wiso.uni-koeln.de